SCALER

Binary Search - 3

TABLE OF CONTENTS

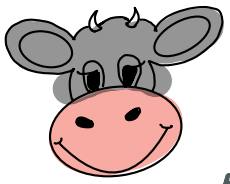
Notes

‘Your smile is your logo, your personality is your business card and how you leave others feeling is a trademark.’

~ Anonymous

< Question > : Farmer has built a bar with N stalls.

A[i] → location of ith stall in sorted order.



no of cows

m = 3

A[ ] → [1, 4, 8, 10]

2 ≤ M ≤ N

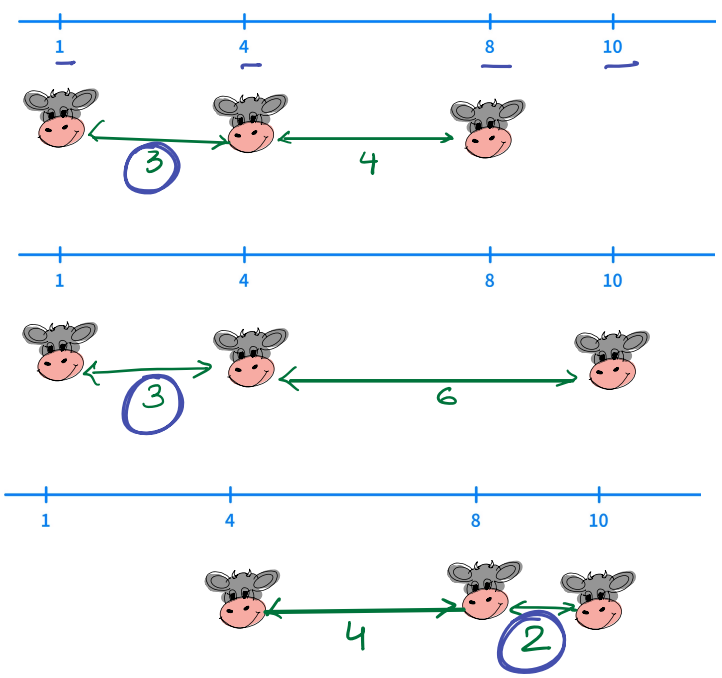
Cows are aggressive towards each other. So, farmer wants to maximise the minimum distance between any pair of cows.

Find max possible minimum distance.

0 1 2 3

{1 4 8 10}

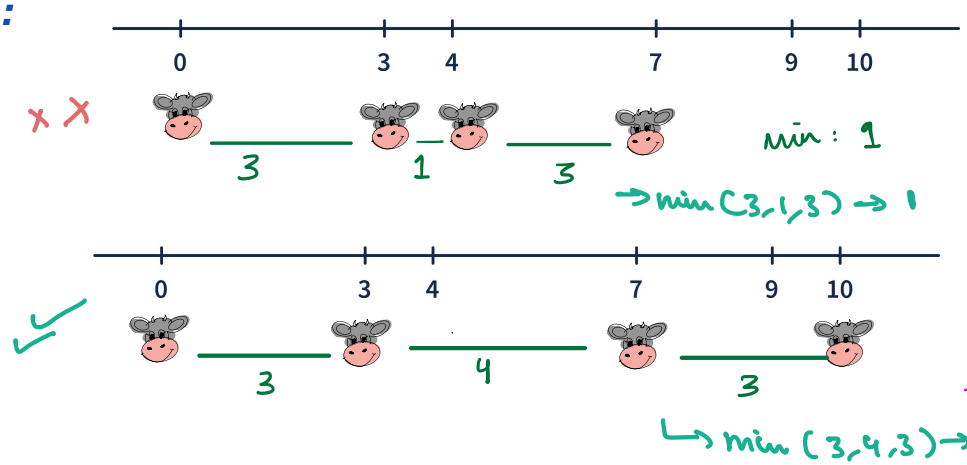
N=4



Ans → 3

the best possible config in which the min distance bet^n 2 cows max can be 3

Quiz :



K =

A = [0, 3, 4, 7, 9, 10]

K = 4

ans: 3

↳ min(3,4,3) → 3

BF Idea

Try out different combinations & check distances.

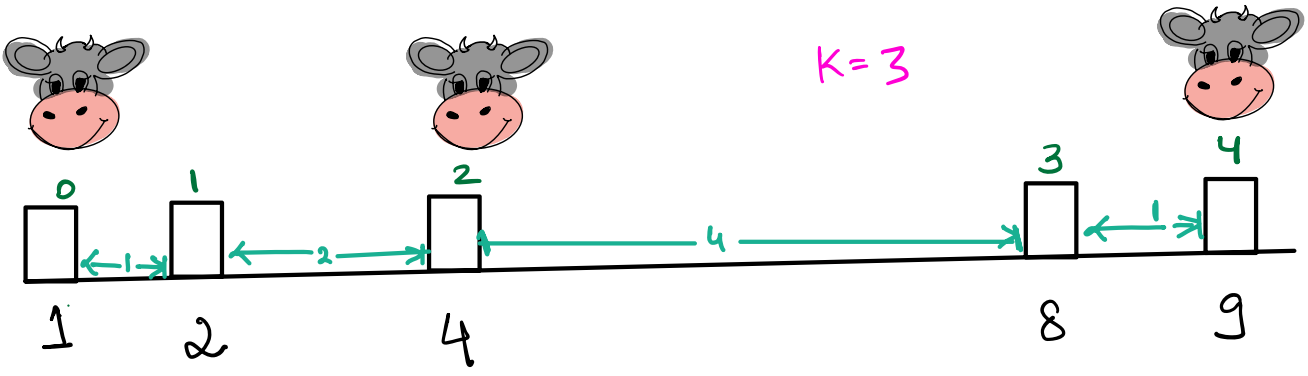
$= {}^{n}C_{K} * n$

Assume : There will be atleast 2 cows. $2 <= K$

what are all possible distances we can have as answer ?

answ Worst possible answer → 1

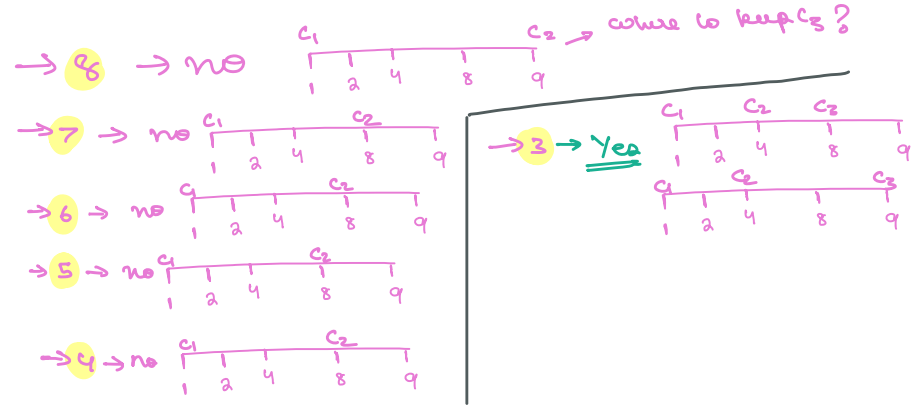
ansb Best possible answer → 8 [Amax − Amin]



take decision what is the max distance bet^n 2 chairs ⇒ 9−1 = 8

↳ can I keep the cows in a way such that min distance bet^n them is k=3

So



for the range of max to min check if

```
for(d = ansb ; d >= answ ; d--){
    if(isValidDis(A, K, d)){
        return d
    }
}
```

```
boolean isValidDis(A[], int K, int d){
    prev_idx = 0
    cows = 1
    for(i = 1; i < N; i++){
        if(A[i] - A[prev_idx] >= d){
            cows++
            prev_idx = i
        }
    }
    if(cows >= K) return true
    return false
}
```

- ansb: 8
- answ: 1
- A[]: array is sorted
- K: # given cows
- d: distance that we are trying
- prev_idx = 0 → stores idx at which last cow is kept
- cows = 1 → this comes from assumption that we have 1st cow at 0th index so that's we started from i=1
- if(A[i] - A[prev_idx] >= d) → to check if we can place cow at that idx if dist ≥ k
- TC: O(range * N), range = Amax - Amin

how to optimise further → Bin Search ??

① do I have sorted search space → range of possible space: 1, 2, 3, 4, 5, 6, 7, 8

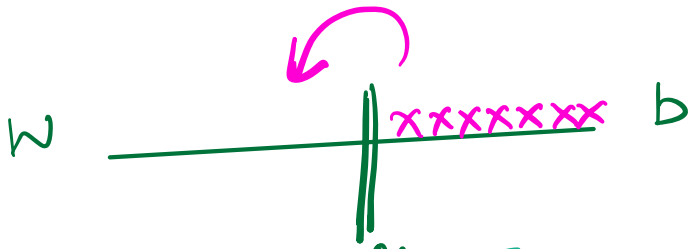
② do I have search value → highest value which justify search space

③ decision to take → right (take a higher value) if distance is valid to search for a bigger ans

↓ left: take a smaller value if distance not valid

where can we apply binary search? → where there is Monotonicity

↳ that is you discard mid then the right or left can also be discarded

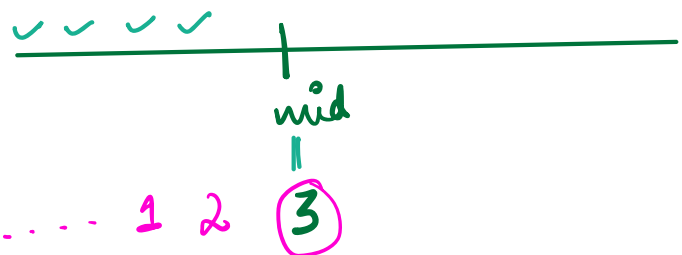


mid -> 5

can't keep at mid. → then you can't keep then at 7, 8, 9 also

5 6 7 8 9 . . . . . x x x

= x x x x



. . . . 1 2 3

looking at mid so we should see for a higher one

Idea -2

Sorted search space → 1 to Amax − Amin

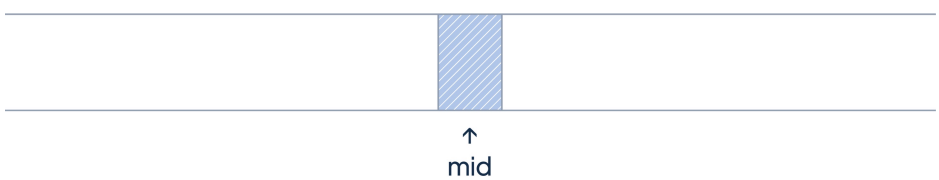
target → placing K cows

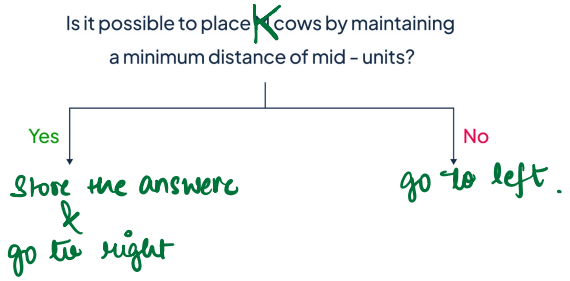
Condition on which search space can be reduced

↳ if (mid → isvalid) {
ans = lo
lo = mid + 1
} else {
ho = mid − 1
}

mid → right (take a higher value) if distance is valid to search for a bigger ans

↓ left take a smaller value if distance not valid

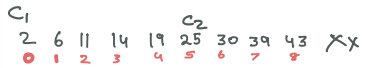
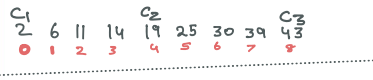
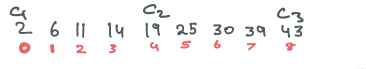
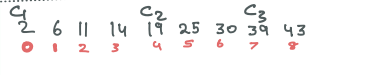




*Dry Run

K = 4

→ suppose the dist^n betⁿ 2 cows

l	r	mid	Can we place K cows with distance ≥ mid?	
1	41	22	go left	
1	21	12	ans = 12 go right	
13	21	17	go left	
13	16	14	go left	
13	13	13	go left	

13 > 12 stop

ANSWER

$A[i] <= 10^9$ (int)

< / > Code

```
sort the array if not already sorted
l = 1   // min possible distance
r = A[N-1] - A[0]   // max possible distance

while ( l <= r){
    mid = l + (r-l)/2
    if ( isValidDis (A, K, mid) ) { //checking whether we can keep cows while maintaining a dist. of >= mid
        ans = mid
        l = mid + 1
    }
    else {
        r = mid - 1
    }
}
return ans
```

TC → $O(\log_2(\text{range}) * N)$

Painter's Partition

Given N boards with length of each board

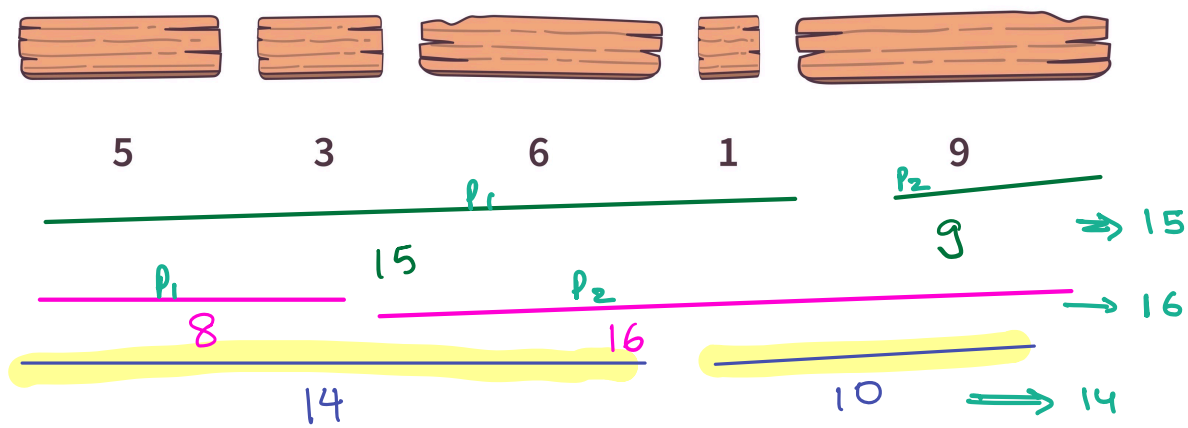
a) A painter takes 1 unit of time to paint 1 unit of length.

b) A board can only be painted by 1 painter.

c) A painter can only paint boards placed next to each other (i.e continuous segment).

< *Question* > : Find minimum time to paint all boards if P painters are available.

N = 5



24

ans: 14 =

Target → min value which can be taken
time taken to complete all boards with min time

Search Space →

min time — max time

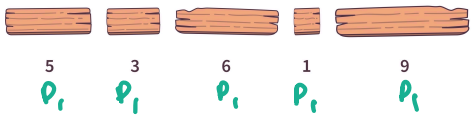
when we have as many painters as the no. of boards

when we have only one painter who has to paint all the boards

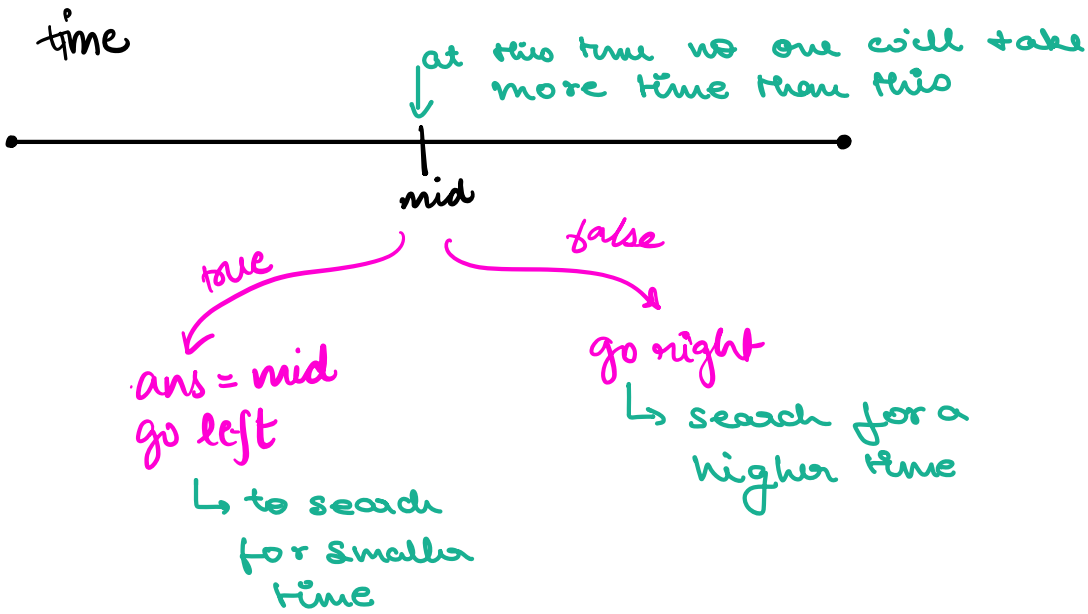
min = Amax → 9

→ Time ⇒ A man

max = Asum = 24



Condition →



when we have to employ more painters than we have, then it'll not work, go right.

Example :

[3 5 1 7 8 2 5 3 10 1 4 7 5 4 6]

K = 4 → check higher time

l (lowest time)	r (largest time)	No one takes more than mid	Is it possible to paint all boards by P painters in time = mid?	
10	71	40 ✓	yes ans=40 go left	[3,5,1,7,8,2,5,3,10,1,4,7,5,4,6] → P_1 | P_2
10	39	24	yes ans=24 go left	[3,5,1,7,8,2,5,3,10,1,4,7,5,4,6]
10	23	16	go right	3,5,1,7 | 8,2,5 | 3,10,1 | 4,7 | 5,4,6 → P_1 P_2 P_3 P_4 ✗
17	23	20	go right	3,5,1,7 | 8,2,5 | 3,10,1 | 4,7 | 5,4,6
21	23	22	yes, ans=22 go left	3,5,1,7,8,2,5,3,10,1,4,7,5,4,6
21	21	21	go right	3,5,1,7,8,2,5,3,10,1,4,7,5,4,6

22 > 21 stop

P_1 → how many board I can give to P_1 so that it does → 36
not exceed mid → say 40 → 38

P_2 → 36

< / > Code

Binary Search Code

// TODO

```
for ( A[] , int P )
lo = A[max]
hi = A[sum]
ans = hi
while ( lo <= hi ) {
    mid = lo + (hi - lo)/2
    if ( canPaint ( A, P, mid ) {
        ans = mid
        hi = mid-1
    } else {
        lo = mid+1
    }
}
```

```
bool canPaint ( A[], P, time ) {
    painters = 1      work_time = 0
    for( int i = 0 ; i<N; i++ ){
        work_time += A[i]
        if ( work_time > time ) {
            painters ++
            work_time = A[i]
        }
    }
    if ( painters <= P ) return true
    return false
}
```

[Amax , Asum]

TC → O(log2(range) * N)

Both problems involve:
* Optimising a certain value (maximising distance or minimising time) using binary search.
* Checking the feasibility of each candidate solution with a linear pass through the data, which typically involves a greedy approach.

This is a useful technique in scenarios where direct computation might be infeasible due to high complexity, and where a property of monotonicity can be exploited (i.e., if a certain "answer" works, all answers greater than this might work as well).

Situation:

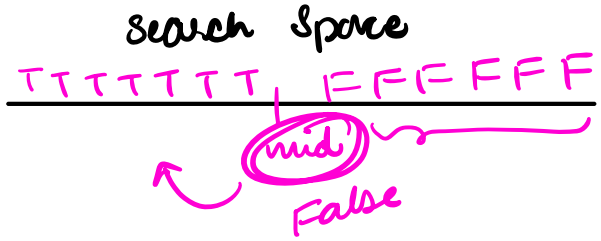
Imagine you are tasked with developing a system for evenly distributing the workload among a team of email response handlers in a customer service department. Each email is assigned a 'complexity score' which represents the estimated time and effort required to address it. The complexity scores are represented as an array, where each element corresponds to a single email.

Task

The goal is to divide the array into K contiguous blocks (where K is the number of email handlers), such that the maximum sum of the complexity scores in any block is minimized. This approach aims to ensure that no single email handler is overwhelmed with high-complexity emails while others have a lighter load.

minimize something
maximize something

Monotonicity



Look eating banana